# CLASIFICADOR DE USUARIOS

Diseño de algoritmo con estructuras condicionales anidadas en Java

**Estudiante:** Jhoymar Daniel Reyes Barrios

**Programa:** Ingeniería Industrial

**Institución:** Corporación Universitaria Americana

**Asignatura:** Fundamentos de Programación

**Fecha:** 30 de octubre de 2026

Informe académico

# 1. Introducción

La programación permite transformar una necesidad cotidiana en una secuencia ordenada de instrucciones que puede ejecutar un computador. Para lograrlo, es necesario identificar los datos de entrada, establecer reglas de decisión, procesar la información y presentar resultados comprensibles. En este trabajo se desarrolla un módulo denominado ClasificadorUsuarios, elaborado en el lenguaje Java, cuyo propósito es clasificar a las personas según su edad y el tipo de suscripción que poseen.

El problema se plantea en el contexto de una plataforma digital que necesita reconocer diferentes grupos de usuarios para asignarles una categoría y validar el nivel de acceso contratado. La edad se organiza en cuatro rangos: niño, adolescente, adulto y adulto mayor. Por otra parte, el tipo de suscripción puede ser gratuita o premium. La combinación de ambos datos produce distintos resultados, por lo que no basta con realizar una sola comparación.

La solución utiliza estructuras condicionales anidadas de tipo if-else. Primero se comprueba que la edad sea válida y se determina la categoría correspondiente. Después, dentro del flujo de una edad válida, se evalúa la suscripción y se informa el acceso. También se contemplan entradas incorrectas, como una edad negativa o un tipo de suscripción diferente de los dos permitidos. Finalmente, se presentan el pseudocódigo, el diagrama de flujo, la implementación, una matriz de doce casos de prueba y recomendaciones para mejorar el módulo.

# 2. Objetivos

Objetivo general: Diseñar e implementar un algoritmo en Java que clasifique usuarios de acuerdo con su edad y tipo de suscripción, aplicando estructuras condicionales anidadas y verificando su funcionamiento mediante casos de prueba representativos.

Objetivos específicos:
• Identificar los requisitos funcionales y los rangos de edad que determinan cada categoría.
• Representar la solución mediante pseudocódigo y diagrama de flujo antes de programarla.
• Implementar la lógica de clasificación con estructuras if-else en Java.
• Comprobar los resultados mediante al menos diez pruebas que incluyan límites y datos inválidos.
• Documentar los resultados y proponer mejoras para futuras versiones del programa.

# 3. Justificación teórica

Las estructuras de decisión permiten que un programa seleccione una acción según el cumplimiento de una condición. En Java, la instrucción if evalúa una expresión booleana: si su resultado es verdadero, ejecuta un bloque; de lo contrario, puede continuar con una alternativa else. La estructura else if permite revisar varias condiciones en orden, mientras que los bloques pueden contener otras decisiones.

Una estructura condicional anidada aparece cuando una decisión se encuentra dentro del bloque de otra decisión. Esta organización es útil cuando una segunda validación depende de que la primera haya sido superada. En el presente ejercicio, primero se valida la edad. Si es negativa, el programa informa el error y no intenta clasificarla. Si es válida, determina el rango etario y luego revisa la suscripción para definir el nivel de acceso.

La necesidad de anidar decisiones se explica porque el resultado depende de dos dimensiones: edad y suscripción. La edad por sí sola no determina si el acceso es gratuito o premium, y la suscripción por sí sola no informa si la persona es niño, adolescente, adulto o adulto mayor. La solución debe considerar ambas variables para producir un resultado completo.

El uso de condicionales anidadas también ayuda a organizar la lógica en etapas. Primero se descartan datos imposibles; después se asigna una categoría; por último, se valida el nivel de acceso. Este orden evita ejecutar reglas que no corresponden a una entrada inválida. Además, permite que el código sea leído y revisado con mayor facilidad cuando se emplean nombres descriptivos y bloques correctamente delimitados.

Java es apropiado para este ejercicio porque ofrece una sintaxis clara para controlar el flujo del programa y cuenta con herramientas de desarrollo ampliamente utilizadas. La clase Scanner permite leer información desde la consola, mientras

que los métodos estáticos facilitan separar la lógica de clasificación de la interacción con el usuario. Esta separación también hace posible probar la lógica sin tener que escribir manualmente los datos en cada ejecución.

Las decisiones anidadas deben diseñarse con cuidado. El orden de las comparaciones de edad es importante: después de descartar valores negativos, se revisa si la edad es menor o igual a 12; si no lo es, se comprueba si es menor o igual a 17; luego si es menor o igual a 59; y finalmente se asigna la categoría de adulto mayor. Como las condiciones se revisan de menor a mayor, cada valor válido queda ubicado en un único rango. Los límites 12, 13, 17, 18, 59 y 60 son especialmente importantes porque representan cambios de categoría.

También es importante validar la suscripción. El programa acepta las palabras gratuita y premium, sin distinguir entre mayúsculas y minúsculas y eliminando espacios al inicio o al final. Si el dato es nulo o contiene otro texto, devuelve un mensaje de suscripción no válida. Esta validación evita que una entrada inesperada se interprete como un nivel de acceso permitido.

## 4. Requisitos funcionales y reglas del sistema

El módulo recibe dos datos: edad, expresada como un número entero, y suscripción, expresada como texto. El sistema debe verificar que la edad no sea negativa. Los rangos definidos son: de 0 a 12 años, niño; de 13 a 17 años, adolescente; de 18 a 59 años, adulto; y desde los 60 años, adulto mayor.

La suscripción tiene dos valores aceptados: gratuita y premium. Si el valor es gratuita, el mensaje final debe indicar acceso gratuito. Si es premium, debe indicar acceso premium. Cualquier otro texto debe generar un mensaje de error. La clasificación de edad y la validación de suscripción son reglas independientes, pero ambas deben cumplirse para obtener una salida completa.

Como requisitos no funcionales básicos, se busca que el programa sea sencillo de ejecutar, que sus mensajes sean comprensibles, que el código esté organizado y que la lógica pueda probarse de manera independiente. El ejercicio no administra cuentas reales ni concede permisos a una plataforma; representa una simulación académica de reglas de clasificación.

## 5. Diseño algorítmico: pseudocódigo y flujo

Antes de escribir el código Java se elaboró un pseudocódigo para describir la secuencia de pasos sin depender de la sintaxis de un lenguaje específico. El algoritmo lee la edad y la suscripción. Si la edad es menor que cero, presenta un mensaje de error. En caso contrario, realiza las comparaciones ordenadas para asignar una categoría. Luego normaliza el texto de la suscripción y verifica si corresponde a gratuita o premium. Si no coincide con ninguna, informa que el tipo no es válido.

El diagrama de flujo representa visualmente la misma lógica. Los paralelogramos corresponden a la lectura de datos y presentación de resultados; los rombos representan preguntas que generan caminos de sí o no; y los rectángulos representan procesos como asignar una categoría. Todos los caminos terminan en el nodo de fin. El diagrama permite observar que la validación de suscripción solo se realiza después de confirmar que la edad es válida.

La representación previa es valiosa porque permite detectar omisiones antes de programar. Por ejemplo, si no se incluye una rama para edades negativas, el programa podría clasificar un dato incorrecto como niño. De igual forma, si no existe una alternativa final para suscripciones desconocidas, el sistema podría dejar al usuario sin una respuesta clara.

## 6. Implementación en Java y entorno de desarrollo

El proyecto se denomina ClasificadorUsuarios y contiene una clase principal con el mismo nombre. Puede crearse en Eclipse, IntelliJ IDEA, NetBeans o cualquier editor compatible con Java. Se requiere instalar un JDK y configurar el IDE para utilizarlo como entorno de compilación y ejecución. Para este proyecto no se necesitan dependencias externas.

La clase contiene el método clasificarUsuario, que recibe la edad y la suscripción y devuelve un mensaje. Esta decisión de diseño separa la lógica del programa de la lectura de teclado. El método puede recibir datos desde la aplicación principal o

desde una clase de pruebas. Así se evita duplicar las reglas y se facilita comprobar distintos escenarios.

En el método, la primera condición verifica si la edad es menor que cero. Cuando se cumple, se retorna inmediatamente el mensaje correspondiente. Si la edad es válida, se declara una variable de texto llamada categoria y se asigna uno de los cuatro grupos mediante if, else if y else. Después se comprueba que la suscripción no sea nula, se eliminan espacios externos y se convierte el texto a minúsculas. Por último, las condiciones comparan el tipo con gratuita y premium.

El método main utiliza Scanner para solicitar los datos al usuario. Antes de leer la edad se comprueba que la entrada sea un entero, con el fin de evitar un error de lectura si se escribe texto. Después se captura la suscripción y se imprime el resultado que devuelve el método. La clase Scanner se cierra al terminar.

La implementación evita que la interacción por consola quede mezclada con la lógica de clasificación. Esta separación es una práctica útil incluso en programas pequeños, ya que permite modificar la forma de entrada de datos sin alterar las reglas principales.

## 7. Pruebas y verificación

Las pruebas permiten comprobar si el programa produce la salida esperada ante diferentes entradas. Para este ejercicio se diseñaron doce casos, superando el mínimo solicitado de diez. Se incluyen edades representativas dentro de cada grupo, valores situados justo en límites de clasificación, una edad negativa y una suscripción no reconocida.

Los casos 1 y 2 comprueban el grupo niño con ambos tipos de suscripción. Los casos 3 y 4 hacen lo mismo para adolescente. Los casos 5, 6 y 7 revisan el grupo adulto, incluyendo el límite superior de 59 años. Los casos 8, 9 y 10 verifican adulto mayor con diferentes edades y suscripciones. El caso 11 comprueba que una edad negativa sea rechazada. El caso 12 verifica que un tipo de suscripción no contemplado no se acepte.

Para ejecutar las pruebas se utiliza la clase ClasificadorUsuariosTest. Esta clase invoca el método clasificarUsuario con cada combinación y compara el resultado obtenido con el esperado. Si ambos textos coinciden, imprime EXITOSO; de lo contrario, imprime FALLIDO y muestra los dos valores para facilitar la corrección. Es una prueba básica automatizada, sin framework externo, adecuada para demostrar la lógica en un ejercicio introductorio.

La tabla de verificación incluida en la carpeta pruebas registra los datos de entrada y los resultados esperados. Al ejecutar la clase de pruebas, el resultado esperado es que los doce casos aparezcan como EXITOSO. Esta afirmación describe el resultado previsto por la lógica implementada; para la entrega académica se recomienda ejecutar efectivamente el programa en el equipo, conservar una captura de la consola y anexarla como evidencia. Si aparece un caso fallido, se debe revisar la condición correspondiente, corregir el código y repetir todas las pruebas.

La prueba de límites es especialmente importante. Una edad de 12 debe clasificarse como niño, mientras que 13 debe clasificarse como adolescente. De manera similar, 17 corresponde a adolescente y 18 a adulto; 59 corresponde a adulto y 60 a adulto mayor. Estos pares permiten detectar errores comunes como utilizar menor que en vez de menor o igual que.

## 8. Resultados esperados y análisis

La ejecución correcta debe producir una clasificación coherente para cada combinación válida. Los usuarios de cualquier rango de edad pueden tener suscripción gratuita o premium; por eso, el sistema no restringe un tipo de suscripción según la edad. La edad define la categoría y la suscripción define el nivel de acceso, de modo que ambas dimensiones se reflejan en el mensaje.

Los casos inválidos se manejan por separado. Una edad negativa impide continuar con la clasificación. Una suscripción desconocida no genera acceso gratuito ni premium. Esta conducta es preferible a asumir un valor por defecto, porque evita conceder un resultado que el usuario no solicitó o que el sistema no reconoce.

La matriz de pruebas no reemplaza la ejecución real. Su función es establecer entradas, resultados esperados y criterios para determinar si la implementación cumple los requisitos. La clase de pruebas automatiza la comparación y hace repetible la verificación. La evidencia final debe corresponder a una ejecución real en el JDK configurado.

## 9. Recomendaciones de mejora

Una primera mejora sería ampliar la validación de entradas. Aunque el programa comprueba que la edad sea un entero desde la consola, una versión más robusta podría controlar valores excesivamente altos y permitir repetir la solicitud cuando el usuario escriba un dato incorrecto. También se podría utilizar un menú para que la persona seleccione la suscripción en lugar de escribirla.

Una segunda mejora sería incorporar pruebas con JUnit. La clase incluida cumple la función de verificación básica y no necesita librerías, pero JUnit permite organizar pruebas unitarias, ejecutar suites y obtener reportes automáticos dentro del IDE. También sería conveniente añadir casos para una suscripción nula, texto con espacios y distintas combinaciones de mayúsculas.

Otra mejora consistiría en separar las categorías y tipos de suscripción mediante enumeraciones (enum), evitando depender de cadenas de texto. En un sistema real, además, las reglas de edad y acceso deberían definirse con el área responsable, documentar excepciones y proteger los datos personales. El programa de este trabajo es una demostración educativa y no debe utilizarse para tomar decisiones reales sobre derechos o servicios.

Finalmente, se recomienda mantener nombres claros, comentarios breves, sangría uniforme y un README con instrucciones de compilación. El repositorio debe incluir únicamente archivos necesarios y una estructura que permita al docente encontrar fácilmente el código, las pruebas y el informe.

## 10. Conclusiones

El desarrollo del ClasificadorUsuarios permitió aplicar estructuras condicionales anidadas para resolver un problema que combina más de una condición. La edad determina una de cuatro categorías y la suscripción establece el tipo de acceso que se informa. La solución demuestra que una decisión puede organizarse en etapas dependientes y que las alternativas else if permiten cubrir rangos sin superponerlos.

El uso de pseudocódigo y diagrama de flujo facilitó planificar el comportamiento antes de escribir el programa. La implementación en Java separó la lógica de clasificación de la lectura por consola, lo que hizo posible probar el método de manera independiente. Asimismo, se contemplaron entradas inválidas para evitar resultados incorrectos.

Los doce casos propuestos cubren las combinaciones principales y los límites de edad más relevantes. La clase de pruebas permite comparar automáticamente el resultado obtenido con el esperado. Como paso final de la actividad, se debe compilar y ejecutar el proyecto en el entorno configurado, verificar que todas las pruebas sean exitosas y guardar evidencias reales para anexarlas al repositorio.

En conclusión, el ejercicio fortalece la comprensión del control de flujo, la validación de datos, la documentación y las pruebas básicas. Estos elementos son fundamentales para desarrollar programas más confiables y fáciles de mantener.

## Anexo A. Matriz de casos de prueba

| N.º | Edad | Suscripción | Resultado esperado |
|---|---|---|---|
| 1 | 8 | gratuita | Niño - Gratuito |
| 2 | 12 | premium | Niño - Premium |
| 3 | 13 | gratuita | Adolescente - Gratuito |
| 4 | 17 | premium | Adolescente - Premium |
| 5 | 18 | gratuita | Adulto - Gratuito |
| 6 | 30 | premium | Adulto - Premium |

| N.º | Edad | Suscripción | Resultado esperado |
|---|---|---|---|
| 7 | 59 | gratuita | Adulto - Gratuito |
| 8 | 60 | premium | Adulto mayor - Premium |
| 9 | 75 | gratuita | Adulto mayor - Gratuito |
| 10 | 100 | premium | Adulto mayor - Premium |
| 11 | -5 | gratuita | Edad no válida |
| 12 | 25 | basica | Suscripción no válida |

Nota de evidencia: ejecutar ClasificadorUsuariosTest en el IDE o terminal y adjuntar una captura de la salida real. No presentar esta tabla como captura de ejecución.